

VIGIL

Real-time fraud detection and prevention for Indian digital lending

Roll number **2022B2A81416G** · Repository <https://github.com/anujkhandelwal-26/vigil->

VIGIL scores every loan application with a LightGBM model in milliseconds, explains each score with SHAP, and lets a retrieval-augmented LLM copilot answer analyst questions strictly from retrieved case data, never from parametric guesswork. Decisions route to one of four actions (Approve / KYC verification / Review / Decline) rather than a binary accept-reject, cutting hard declines without letting fraud through.

1. Approach

The brief asks for four capabilities. Each is demonstrated in the running system, not only described:

Requirement	How it is met
Real-time	POST /api/v1/applications answers in a measured p95 of 99ms end-to-end across 200 sequential requests through the full stack; the LLM is never on this path.
Self-learning	POST /api/v1/models/retrain (admin only) rebuilds the model from the dataset plus every analyst_feedback row at higher sample weight, and only hot-swaps if the new holdout PR-AUC is at least the incumbent's. One confirmed case moved a held-out fraud pattern's score from 0.000 to 0.714 after a single retrain.
Fewer false positives	Four actions instead of two: STEP_UP (shown to analysts as KYC) and REVIEW absorb the grey zone a binary threshold would hard-decline. Binary decline 3.48% versus four-way 3.46%; binary FP rate 0.023% versus four-way 0.012%.
New fraud vectors	An IsolationForest novelty channel runs alongside the supervised model, deliberately not blended into the risk score. The held-out “new bot ring” scenario scores risk 0.000 but anomaly 0.949: the supervised model misses it entirely, the novelty channel catches it.

Applications sharing a device, bank account or mobile hash — or merely a similar behavioural signature with every identifier rotated — are grouped by pgvector cosine-distance clustering into a fraud_ring record, surfaced on the case detail view with member count and detection method.

LightGBM was chosen over a deep model because tabular data at this size (60k rows, ~40 features) is its strength: it trains in seconds, which is what makes retrain-on-feedback practical, and it yields exact SHAP attributions without a separate explainability pipeline. The LLM is restricted to rendering prose from a payload that has already been retrieved and decided — it never produces a score, a decision or a reason code, and training runs on synthetic data because three public benchmarks were evaluated and rejected on evidence, not assumption (Section 2).

2. Key insights and findings

Why synthetic data, not a public dataset

Three public fraud benchmarks were evaluated before committing to a synthetic generator, and all three were rejected on evidence rather than assumption: one had no identity or timestamp fields and several columns statistically indistinguishable from noise; one had a post-outcome field that leaked the label outright (near-perfect accuracy for the wrong reason); and the strongest candidate carried a self-contradictory non-commercial licence, a documented prior label leak, and modelled a different, non-Indian task. None offered the entity-linkage fields a ring-detection story needs. A further data point counted against generative synthesis specifically: published results show a fraud model's detection rate dropping by roughly 21 points after GAN-based synthesis, i.e. that style of synthesis destroys the signal it is meant to preserve. Rule-based, structural synthesis was used instead for exactly that reason (full evaluation notes: docs/dataset-audit.md).

How the synthetic dataset is built, and why each block exists

60,000 applications are generated across eight data blocks that mirror what an Indian digital lender actually collects at application time, so that every field maps to a concrete, explainable reason it might indicate fraud rather than being an arbitrary numeric column:

Block	What it captures and why it signals fraud
Applicant	Age, employment type, income, city tier and a 3-digit pincode prefix. Very young/old applicants and an unemployed or student profile applying for a large loan are atypical; income is the baseline for affordability and income-mismatch checks; the typology mix itself differs by city tier. Gender and age are captured but deliberately never given to the model — they exist only for the post-hoc fairness audit.
Loan	Product, amount, tenure and channel. Consumer-durable and POS-channel loans see the most device-farm fraud; larger tickets raise the incentive to commit fraud in the first place; app-based channels see more bot/emulator activity than assisted POS channels.
Bureau	CIBIL score, active loans, enquiries in the last 30 days, delinquency and utilisation. A null or thin file paired with a high requested amount is a classic synthetic-identity flag; a burst of enquiries across lenders is a textbook fraud-ring signal; near-100% utilisation on a brand-new application is a stress signal.
KYC	PAN format validity, PAN name-match score, Aadhaar-PAN linkage, name/DOB consistency and DigiLocker document fetches — mapped directly to Indian regulatory identity checks. A low name-match score is a possible synthetic identity; an unlinked Aadhaar-PAN pair is a flag under RBI KYC norms.
Device	A salted device hash plus reuse count, emulator/root flags, install age, IP prefix and VPN/proxy use. Emulated or rooted devices are near-exclusively fraud; a device reused across many applicants is the core device-farm signal; a fresh install used immediately for a large loan, or many applications from one IP prefix, both point to a ring. Hashes are salted identifiers used only for entity linkage and pgvector clustering, never raw model input.
Behaviour	Form-fill time, typing speed, paste events, field corrections and session screens — telemetry that separates a human filling out a form from a script. Sub-normal fill time, implausible typing speed, or zero corrections at high speed all suggest an automated submission rather than a person. This block also drives the 768-dimension behavioural embedding used to catch rings that share a pattern but rotate every identifier.
Banking	A salted bank-account hash, penny-drop name-match, account age, average monthly credit and salary regularity. A low penny-drop match indicates a mule or rented account; salary credits that don't match a declared “salaried” status, or repeated bounces, indicate income inflation. How many applicants share one account is tracked for ring analysis only — deliberately not fed to the model raw, since it would leak the answer.
Mobile	Mobile-network age, a recent SIM-swap flag and mobile name-match — the core signals for SIM-swap account takeover: a number that is brand-new on the network, or was recently swapped, immediately used for a large loan.

Each of the five training typologies plants correlated signals across several of these blocks at once — a device-farm case looks wrong on device reuse and form-fill speed together, not on one field in isolation — and the signal is injected probabilistically, at blended severity, rather than as a maximal, always-extreme value. That is a deliberate choice: a deterministic “always extreme” signal is trivially separable and is the same failure mode found in the rejected public datasets. A sixth typology, a bot ring on a previously unseen vector (clean bureau file, sub-second form fill, a headless device fingerprint, one device behind many identities), is generated the same way but held out of training entirely, existing solely to demonstrate that the novelty channel catches what the supervised model was never shown.

A plausible default threshold was wrong, and testing the unhappy path caught it

Ring-detection cosine distance for two members of a real ring measures ≈ 0.0002 , even after every identifier is rotated; two genuinely unrelated clean applicants measure ≈ 0.15 – 0.16 ; a borderline legitimate profile measures ≈ 0.008 . An initial threshold of 0.15 (a reasonable prior for sentence embeddings) grouped every application submitted close in time into one ring, including unrelated clean applicants. It was recalibrated to 0.005 — an order of magnitude above true ring cohesion and comfortably below the nearest borderline case — and re-verified against two synthetic rings and three unrelated applicants, all classified correctly. The lesson generalises: a plausible default for a vector-similarity threshold is not a substitute for measuring it against the actual embedding model and data.

Keeping the LLM off the hot path is the single most important design decision

A warm round-trip to the local LLM (qwen2.5:3b) measures ≈ 0.4 s — far too slow for a real-time scoring budget. So the 768-dimension behavioural embedding runs asynchronously right after the HTTP response is sent, and the LLM narrative or copilot only runs on explicit analyst action, cached after first use. This is what lets the scoring path credibly claim real-time performance while the product still offers LLM-generated prose elsewhere.

No single feature exceeds 0.70 univariate AUC on its own; the model's near-ceiling PR-AUC (0.9962) comes from combining roughly fifteen weakly-informative signals per typology, the way a real fraud-ops rule engine would, not from one feature encoding the answer. The model is deliberately strong on the five known typologies so that its failure on the sixth, held-out typology (scores 0.18–0.24, all would-be approvals) reads as a genuine blind spot for the novelty channel to catch, not a sandbagged demo.

3. Proposed solution and architecture

Layer	Technology	Responsibility
Frontend	React 18, Redux Toolkit, RTK Query · :5174	Application dashboard for scenario submission and an analyst dashboard with queue, case detail and copilot.
Backend	Spring Boot 4.1.1, Java 21 · :8081	API-first front door: JWT auth, bean validation, server-side velocity features, a deterministic rule engine, ML orchestration with a timeout-bounded fallback, persistence and audit log.
AI layer	FastAPI, Python 3.12 · :8001	Stateless inference sidecar: LightGBM score, SHAP explanation, IsolationForest novelty, embeddings, pgvector retrieval, the RAG copilot and its guardrails.
Database	PostgreSQL 16 with pgvector · :5433	Structured data (applications, decisions) and vector data (case embeddings, policy corpus, reason-code whitelist) in one database.
LLM / embeddings	Ollama locally; swappable to AWS Bedrock or Anthropic by config only	Narrative generation and the RAG copilot, entirely off the scoring hot path.

Java is the mandated, API-first backend; Python is a stateless inference sidecar Java calls over HTTP with a short, enforced timeout — independently testable and independently scalable, the boundary production fraud systems are built on.

Request path, target p95 under 150ms

JWT verify and validation $\approx$ 2ms; velocity SQL across device, IP, bank and mobile, server-computed and not trusted from the client, $\approx$ 8ms; the rule engine (about fifteen deterministic rules that can only escalate an action, never downgrade one the model already raised) under 1ms; ml-service scoring, with a 150ms timeout that degrades gracefully to rules-only on failure, 30–55ms; persisting the decision $\approx$ 5ms. Measured end to end: p50 $\approx$ 60ms, p95 $\approx$ 99ms. Deliberately kept off this path: the behavioural embedding, computed asynchronously after the response, and the LLM narrative or copilot, generated only on explicit analyst action.

Why pgvector is load-bearing

It grounds three things: similar-case retrieval for analyst decision support and for the copilot's context, so the LLM cannot invent precedent; policy retrieval, so a copilot answer cites the actual RBI, DPDP or KYC passage; and the copilot's intent router, which embeds the analyst's question and matches it against labelled examples per intent. Entity lookup (“all cases on this device”) stays plain SQL deliberately — exact identifier joins are the right tool for that question, not a vector search. If ml-service does not respond within its timeout, the caller falls back to a rules-only decision rather than propagating an error; verified directly by replaying 200 sequential requests over a small pool of shared hashes and confirming later requests correctly escalate to review or decline as the server-observed reuse count crosses threshold.

4. Data, model and results

60,000 synthetic Indian digital-lending applications, 3.5% fraud base rate, five training typologies (synthetic identity, mule account ring, device farm, SIM-swap takeover, income inflation), generated by a rule-based generator rather than a public dataset or a GAN. A sixth typology, a bot ring on a new vector, is held out of training entirely to demonstrate adaptation to a pattern the model has never seen. Trained on a time-based split, never random, since a random split would scatter ring members across train and test and inflate reported performance. Gender and age are stored but never used as model features — only for the post-hoc fairness audit below.

Metric	Value
Algorithm	LightGBM, isotonic-calibrated, regularised against memorising the synthetic signal
PR-AUC / recall at 1% FPR (test)	0.9962 / 1.0000
Binary policy decline / FP rate	3.48% / 0.02%
Four-way policy decline / FP rate	3.46% / 0.01%
Fairness — disparate-impact ratio	0.994–1.000 across gender and age bands (80% rule satisfied with room to spare)

Four-way thresholds are grid-searched to minimise expected cost in rupees (loss on a missed fraud INR 150,000; cost of a wrongly declined good customer INR 8,000; step-up INR 150; review INR 400), resulting in thresholds of 0.05, 0.75 and 0.80. Every decision carries SHAP's top five contributing features, mapped to a whitelisted, regulator-shaped adverse-action reason code under ECOA/Regulation B and RBI fair-practice expectations.

What actually drives a score

SHAP's TreeExplainer was run against the trained LightGBM model over an 800-row sample of the held-out test set, ranking every feature by its mean absolute contribution to the model's output. The result confirms the design intent from Section 2: the largest contributors are cross-block consistency checks — declared income against actual bank credit, salary regularity, PAN name-match — rather than any single raw field, which is exactly why no individual feature reaches 0.70 univariate AUC on its own while the combined model does.

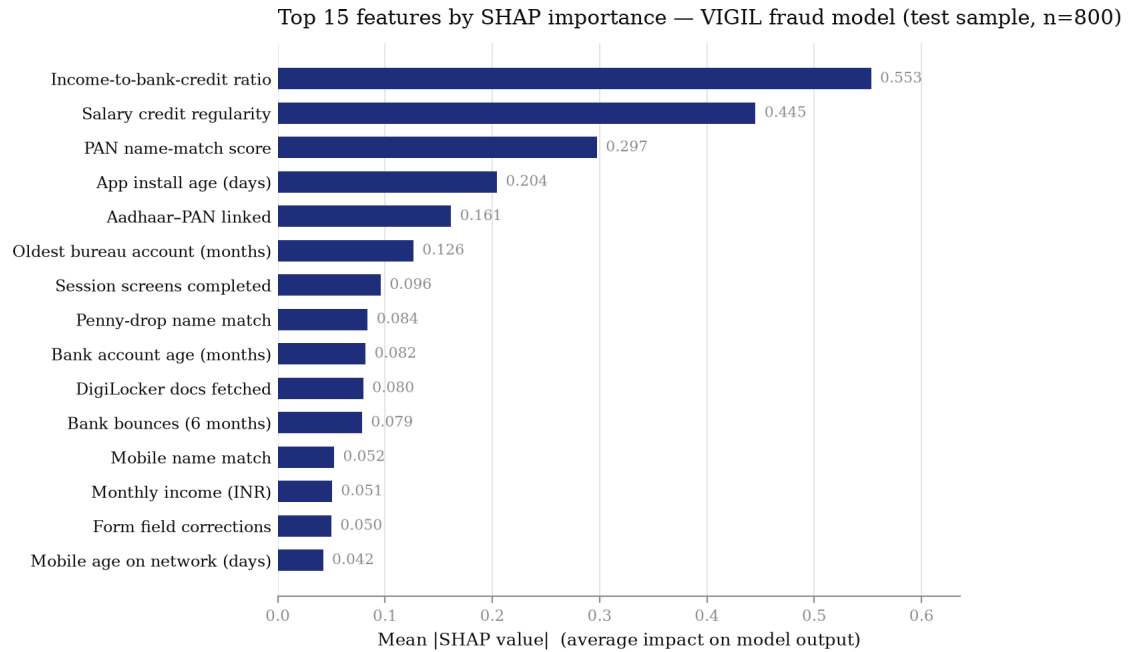


Figure 1. Mean |SHAP value| for the top 15 features, computed with `shap.TreeExplainer` against the trained model (`services/ml-service/artifacts/model.pkl`) on a sample of the test set.

5. Cloud layer: the AWS target architecture

VIGIL runs locally for the demo (docker-compose Postgres, native processes for the three app services) to avoid container-networking risk close to the deadline, but is designed against a concrete AWS deployment target:

Local	AWS target	Why
docker-compose Postgres	RDS PostgreSQL, Multi-AZ, pgvector	Managed failover and backups; pgvector ships as a supported RDS extension.
spring-boot:run / uvicorn	ECS Fargate, one task family per service, behind an ALB	No server management; scales on load; matches the existing service boundary.
.env file	Secrets Manager via ECS task IAM role	No secret ever touches a container image or source control.
Ollama	AWS Bedrock (Claude for narrative/copilot, Titan Embeddings v2)	Same LlmProvider/EmbeddingProvider interface already implemented; switching provider is a config change.
Vite dev server	CloudFront over an S3-hosted build	Static asset delivery at the edge.
—	APIGEE or API Gateway, plus AWS WAF, in front of the ALB	Rate limiting, API keys, quota, and standard managed rule sets.
Local model artifacts	S3, versioned, referenced by the model registry	Retraining writes a new artifact and registry row; the active model swaps by config.

What would change in the code for this migration is almost nothing: the Bedrock provider implementations already exist behind the same interface as the Ollama ones used in the demo, so moving providers is an environment-variable change, not a rewrite — though they are structurally correct against each API's documented shape and have not yet been exercised against live credentials.

6. Security and responsible AI

Control	Implementation
Authentication / authorization	JWT (HS256), bcrypt hashes; role-based (analyst/admin) with method-level enforcement on the retrain endpoint specifically.
Input validation	Bean validation on every field, RFC 7807 error responses.
No hardcoded secrets	Everything from environment variables; .env is gitignored and verified clean by a repo-wide grep sweep.
Responsible data handling	Aadhaar never stored beyond the last four digits; device, bank and mobile identifiers salted and hashed; pincode truncated to 3 digits, IP to /16; PII never enters an LLM prompt.
Secure API usage	CORS locked to the dev origin; the inference service is unreachable from the browser; a 150ms timeout is enforced on the scoring call.

The LLM never decides: it renders prose from a payload already scored and decided. Every LLM output is guardrail-validated before an analyst sees it — reason codes must be on the whitelist, every number must trace back to retrieved context, no decision verbs, no raw PII pattern — and a violation discards the output for a deterministic template rather than letting the LLM police itself. Empty retrieval returns a refusal, never a parametric guess.

7. Engineering practices

Practice	Evidence
Clean, modular code	Three independently owned services with one enforced HTTP boundary between the Java API and the Python sidecar.
API-first design	decision-api is the sole front door; ml-service is called only server-to-server.
Version control	Full history on GitHub: https://github.com/anujkhandelwal-26/vigil-
README and setup instructions	Rationale, architecture, provider swapping, quick start, dataset, API reference, security and testing, all documented.
Architecture diagrams	Request-path latency budget and service diagrams in docs/architecture.md and docs/aws-target-architecture.md.
Test coverage	30 tests, 11 Java and 19 Python, all passing (Section 8).
Secure credential handling	No secret in source control anywhere; AWS target uses Secrets Manager and IAM roles, never static keys.

Practice	Evidence
Responsible AI and explainability	LLM-never-decides invariant, guardrail validation, fairness audit, SHAP-based regulator-shaped reason codes.

8. Testing and verification

30 tests, 11 Java and 19 Python, all passing: rule-engine boundary tests, ML-client degradation and input-validation rejection on the Java side; LLM guardrails, feature-contract determinism, copilot refusal behaviour and pgvector ring detection on the Python side. For this submission, 200 sequential requests were run through the full stack (React, Spring Boot, FastAPI, Postgres): p50 = 66.5ms, p95 = 99.2ms, p99 = 165.1ms, max = 438.9ms. A repo-wide secrets sweep (grep for key/secret/password/token literals, excluding .env and example files) returns nothing. All three Dockerfiles were build-tested and run-tested, not just written — one real bug (a missing libgomp1 runtime dependency for LightGBM) was caught and fixed this way.

9. Repository layout and API reference

vigil/ — README, docker-compose.yml, .env.example · docs/ (architecture, aws-target-architecture, model-card, dataset-audit) · db/ (schema, reason codes, policy corpus) · services/decision-api (Spring Boot / Java) and services/ml-service (FastAPI / Python) · data/generator (synthetic data) · web (React dashboards).

Method	Path	Purpose
POST	/api/v1/auth/login	Issue a JWT
POST	/api/v1/applications	Submit and score an application (the hot path)
GET	/api/v1/applications/{id}	Application plus latest decision
POST	/api/v1/applications/{id}/narrative	Generate or fetch the cached LLM case explanation
POST	/api/v1/applications/{id}/feedback	Analyst verdict feeding the retrain loop
GET	/api/v1/applications/{id}/ring	Fraud-ring membership, if any
POST	/api/v1/copilot/query	RAG analyst copilot
GET	/api/v1/models/cost-curve	False-positive reduction metrics and the fairness audit
POST	/api/v1/models/retrain	Admin-only self-learning retrain with a promotion gate

10. Limitations and future work

- Trained entirely on synthetic data — production deployment needs retraining on real, consented applicant data first.
- Recall near 1.0 at 1% FPR reflects the designed separability of the five known typologies, not a claim about real-world detection performance.
- The novelty channel's 2% contamination rate is a starting assumption, not calibrated against real base rates.
- Bedrock and direct-Anthropic providers are structurally correct but untested against live credentials.
- Similar-case retrieval uses an exact cosine scan, correct and fast at this scale; an ANN index would be the next step at materially larger volumes.

Source, commit history and setup instructions: <https://github.com/anujkhandelwal-26/vigil> · Roll number 2022B2A81416G